

Contents

Bryck REST API <code>session.py</code> and <code>bryck_api.py</code>	1
1. <code>session.py</code>	1
2. <code>bryck_api.py</code>	4
3. <code>ssh_runner.py</code>	14

Bryck REST API `session.py` and `bryck_api.py`

This directory contains a standalone, dependency-free Python client for the Bryck REST API. It is split into two files:

File	Purpose
<code>session.py</code>	Low-level HTTP + JWT session (<code>ApiSession</code>)
<code>bryck_api.py</code>	High-level, one-method-per-endpoint façade (<code>BryckApi</code>) built on top of <code>ApiSession</code>

Both files are pure Python (`requests` + `urllib3` + `stdlib`) and have no dependency on the wider backend/ package. They can be copied and reused verbatim.

- Python: **3.10+** (uses `str | None` unions)
- External deps: `requests`, `urllib3`

1. `session.py`

Purpose

Provide a minimal HTTP session that:

1. Authenticates to `/api/auth` with a username/password.
2. Extracts the JWT token from the login response.
3. Attaches the header `Authorization: JWT <token>` to every subsequent request.
4. Offers generic `get()` / `post()` helpers with optional retry.
5. Can be built directly from a `login.json` config file.

It mirrors the signature of `backend/system_connectors/restful_client.py` so it is a drop-in replacement outside of this project.

Class: ApiSession

```
from session import ApiSession
```

Constructor

```
ApiSession(
    host: str,                                # IPv4 address (hostnames rejected)
    port: int | None = None,                  # defaults: 80 (http) / 443 (https)
    scheme: str = "http",                    # "http" or "https"
    username: str = "admin",
    password: str = "admin",
    timeout: int = 30,                        # per-request timeout in seconds
    max_retries: int = 3,                    # retries for _request_with_retry
    verify: bool | str = False,              # SSL verification (False, True,
        or CA path)
) -> None
```

- Validates host via `ipaddress.IPv4Address` hostnames such as `localhost` raise `ValueError`.
- Auto-corrects known scheme/port mismatches: `https + 80 443`, `http + 443 80` (logged at INFO). Any other combination is passed through untouched.
- Builds `self.base_url = f"{scheme}://{host}:{port}"`.
- Disables `InsecureRequestWarning` when `verify=False`.
- Creates an internal `requests.Session()`.
- Initialises `self.ssh_username` / `self.ssh_password` to `None`; these are populated by `from_login_json` when `bryckserver_*` keys are present. `SshRunner.from_session(session)` reads them.

Factory: from_login_json(path)

```
ApiSession.from_login_json("login.json") -> ApiSession
```

- **Input:** path to a JSON file with the following keys:

Key	Required	Default
<code>bryckapi_host</code>	yes	(must be IPv4)
<code>bryckapi_port</code>	no	80 (auto-corrected against scheme)
<code>bryckapi_scheme</code>	no	"http"
<code>bryckapi_username</code>	no	"admin"
<code>bryckapi_password</code>	no	"admin"
<code>timeout</code>	no	30
<code>bryckserver_username</code>	no	None (required for SSH-using runners)
<code>bryckserver_password</code>	no	None (required for SSH-using runners)
- **Output:** a configured `ApiSession` (not yet logged in call `.login()` next). `bryckserver_username` / `bryckserver_password` are stashed on the instance as `ssh_username` / `ssh_password` for later consumption by `SshRunner.from_session(...)`.

Example `login.json`:

```
{
  "bryckapi_host": "192.168.6.32",
  "bryckapi_scheme": "http",
  "bryckapi_port": "80",
  "bryckapi_username": "admin",
  "bryckapi_password": "<password>",
  "timeout": 300,
  "bryckserver_username": "bryck",
  "bryckserver_password": "<password>"
}
```

Methods

Method	Input	Output	Notes
login()		dict (parsed login response)	POSTs {username, password} to /api/auth, stores token, sets Authorization: JWT <token> header. Raises Exception if no token in response, HTTPError on 4xx/5xx.
get(api_path, params=None, retry=False)	api_path: str (e.g. "/api/config/info"), params: dict\ None, retry: bool	requests.Response	GET with JWT header + timeout. retry=True uses exponential backoff.
post(api_path, payload=None, retry=False)	api_path: str, payload: dict\ None, retry: bool	requests.Response	POSTs JSON body with JWT header + timeout.
close()			Closes the underlying requests.Session.
__enter__ / __exit__		ApiSession / bool	Enables with ApiSession(...) as sess: close() is called on exit. login() is not called automatically.

Method	Input	Output	Notes
<code>address</code> (property)		<code>str</code>	Returns <code>self.base_url</code> .

Retry semantics Only the `_request_with_retry()` helper (invoked when `retry=True`) retries on `requests.exceptions.ConnectionError` with exponential backoff (`2**attempt` seconds), up to `max_retries` attempts. HTTP 4xx/5xx responses are **not** retried they raise immediately.

Auth model

- Header used: `Authorization: JWT <token>` (note the JWT prefix, not Bearer).
- The token is read from either `token` or `access_token` in the login response body.

Minimal usage

```
from session import ApiSession

with ApiSession.from_login_json("login.json") as sess:
    sess.login()
    resp = sess.get("/api/config/info")
    print(resp.json())
```

2. bryck_api.py

Purpose

Wrap every REST endpoint the Bryck exposes as a normal Python method, so callers never have to build URLs or JSON payloads by hand. It also provides one module-level utility used by the runner scripts:

- `ticker(callback, timeout)` polling loop

All endpoints share a single class, `BryckApi`, which takes an authenticated `ApiSession` in its constructor.

For remote shell / SFTP work against the Brycks OS (used by the `format/mount/scan/erase` runners), see `ssh_runner.py` below.

Module-level enums

```
class TaskState(Enum):
    COMPLETED = 2
    ACTIVE = 1
    STALE = 0
    FAILED = -1

class TaskType(Enum):
    TRANSFER = 0
    VERIFICATION = 1
    CAPTURE_BRYCK_STATE = 2
```

Module-level helpers

ticker(callback, timeout) Poll callback() once per second until it returns truthy.

- **Input:** callback: Callable[[], bool], timeout: int (seconds)
- **Output:** None
- **Raises:** TimeoutError if callback does not return truthy within timeout seconds.

Class: BryckApi

```
from bryck_api import BryckApi
api = BryckApi(session=my_session, name="bryck-1")
```

Constructor

```
BryckApi(session: ApiSession, name: str | None = None) -> None
```

- **session must already be logged in** call session.login() first.
- name is an optional label used only for logging.

Central dispatcher All public methods route through:

```
_call(method: str, url_path: str, data: dict | None = None, **kwargs) ->
    Response | None
```

which converts HTTPError, ConnectionError, Timeout, and RequestException into a logged error + None return value. So every public method may return None on failure callers should check.

URL prefixes

Attribute	Value
<code>_cfg_prefix</code>	<code>/api/config/</code>
<code>_download_prefix</code>	<code>/api/download</code>
<code>_network_prefix</code>	<code>/api/network/</code>
<code>_external_storage_prefix</code>	<code>/api/external_storage/</code>
<code>_bcloud_prefix</code>	<code>/api/bcloud/</code>
<code>_tasks_prefix</code>	<code>/api/tasks/</code>
<code>_application_prefix</code>	<code>/api/application/</code>
<code>_settings_prefix</code>	<code>/api/settings/</code>

Endpoint reference

Every method accepts UUIDs as either a single `str` or a `list[str]` (normalized by the internal `_as_list()` helper). Return type is `Response` | `None` unless noted otherwise.

Bryck management (`/api/config/`, `/api/download`)

Method	HTTP	Endpoint	Returns	Purpose
<code>get_hardware_info()</code>	GET	<code>/api/config/info</code>	<code>dict</code> <code>None</code>	Raw hardware info payload.
<code>bryck_info()</code>	GET	<code>/api/config/info</code>	<code>dict</code> <code>None</code>	The result sub-dict only (contains <code>bryck_info</code> , <code>logical_cards</code> , etc.).

Method	HTTP	Endpoint	Returns	Purpose
format_bryck(uuids, store_type, raid_level=0, key_file=None, acfs=None, suffix=None, iqn=None, description="", mountonreboot=False, IoSize=None, DataSync=None, encryption_option=None, compress=None, dedup=None, filestore=True, obj=False, filesystem="zfs", num_vols=None)	POST	/api/config/update	dict \ None	Format logical card(s). Auto-sets encryption_check = bool(key_file or encryption_option). For store_type="BLOCK_STORE" also sends acfs, suffix, iqn.
erase(uuids)	POST	/api/config/reset_store	Response \ None	Re-initialize store(s).
eject(uuids, no_fs_check=None)	POST	/api/config/eject	dict \ None	Eject logical card(s).
mount(uuids, mount_point, key_file, mountonreboot=False, force_check=False, encryption_option=None)	POST	/api/config/mount	Response \ None	Mount logical card(s) at mount_point. force_check is sent as force_mount.
tray_info()	GET	/api/config/tray_info	Response \ None	Tray info.
server_info()	GET	/api/config/server_info	Response \ None	Server info.
shutdown()	POST	/api/config/shutdown	Response \ None	Shut down the Bryck.
upgrade()	POST	/api/config/upgrade	Response \ None	Upgrade firmware.
scan(uuids)	POST	/api/config/scan	Response \ None	Scan logical card(s).
remove(uuids)	POST	/api/config/remove	Response \ None	Remove logical card(s).

Method	HTTP	Endpoint	Returns	Purpose
get_client_package(package_type)	GET	/api/download?name=Response_ClientType	Response_ClientType	Download the client installer (deb, rpm, ...).
download_bryck_report()	GET	/api/download?name=Response_report	Response_report	Download diagnostic report.
download_cloud_transfer_log(transfer_id)	GET	/api/download?name=Response_log&transfer_id=	Response_log	Stream a cloud transfer log.

Object storage (/api/config/...)

Method	HTTP	Endpoint	Purpose
list_object_ip()	GET	list_object_ip	List object-store IP interfaces.
add_object_ip(interface)	POST	add_object_ip	Add an object-store IP interface.
create_object_store_bucket(bucket_name=None)	POST	create_bucket	Create a bucket.
delete_object_store_bucket(bucket_name=None)	POST	delete_bucket	Delete a bucket.
get_object_store_bucket()	GET	list_bucket	List buckets.
create_object_store_access_key(access_key=None, secret_key=None)	POST	create_key	Create S3-style keys (auto-generated when None).
delete_object_store_access_key(access_key)	POST	delete_key	Delete a key.
get_object_store_access_key_list()	GET	list_keys	List keys.

NTP, logs, alerts, email (/api/config/...)

Method	HTTP	Endpoint	Purpose
configure_ntp(uuids, ntp_server)	POST	configure_ntp	Set NTP server on given cards.
marklog(id=None, all=None)	POST	marklog	Mark log entries read.
getlogs(cursor=None)	GET	getlogs	Paged log fetch.
alert_user(user, mailid, alert_type)	POST	alert_user	Add alert recipient.

Method	HTTP	Endpoint	Purpose
<code>alert_user_delete(mailid)</code>	POST	<code>alert_user_delete</code>	Remove alert recipient.
<code>alert_user_list()</code>	GET	<code>alert_user_list</code>	List alert recipients.
<code>config_email_sender(email_type, email_id, email_pass, smtp_url, smtp_port, imap_url, imap_port)</code>	POST	<code>config_email_sender</code>	Configure the outgoing email account.
<code>list_email_sender()</code>	POST	<code>list_email_sender</code>	List configured senders.
<code>del_email_sender()</code>	POST	<code>del_email_sender</code>	Delete configured sender.

Network (/api/network/)

Method	HTTP	Endpoint	Returns	Purpose
<code>network_info(uuids)</code>	GET	<code>info</code>	<code>dict \ None</code>	Returns {uuid: interface_info} filtered to the supplied UUIDs.
<code>configure_network(uuids, interface_name=None, dhcp=None, ip=None, netmask=None, gateway=None, nameservers=None, ntp_server=None, mtu=None)</code>	POST	<code>configure</code>	<code>Response \ None</code>	Configure interface(s). Payload keys: uuids, interface_name, dhcp, ip, netmask, gateway, nameservers, ntp_server, mtu.

Settings (/api/settings/)

Method	HTTP	Endpoint	Returns	Purpose
set_date(option, date=None, time=None, ntp_server=None)	POST	set_date	Response \ None	Set system date/time. option = "Manual" (uses date MM/DD/YYYY + time HH:MM:SS) or "NTP" (uses ntp_server).

NFS external storage (/api/external_storage/)

Method	HTTP	Endpoint	Purpose
nfs_mount(uuids, host, export_path, mount_point)	POST	mount	Mount an NFS export (host is sent as remote_address).
nfs_unmount(uuids, mount_point)	POST	unmount	Unmount an NFS export.

Cloud (/api/bcloud/)

Method	HTTP	Endpoint	Purpose
configure_cloud(bcloud_type, username=None, keyid=None, region=None, keyfile=None, tenant_id=None)	POST	config	Register a cloud provider.
get_cloud_config_list()	GET	config_list	List cloud configurations.
remove_cloud_config(bcloud_type)	POST	config_remove	Delete a cloud configuration.
initiate_cloud_transfer(bcloud_type, src, dst)	POST	transfer	Start a cloud transfer.
pause_cloud_transfer(transfer_id)	POST	pause_transfer	Pause a transfer.
resume_cloud_transfer(transfer_id)	POST	resume_transfer	Resume a transfer.

Method	HTTP	Endpoint	Purpose
cancel_cloud_transfer(task_id)	POST	cancel_transfer	Cancel a transfer.
get_cloud_transfer_status(task_id)	GET	status_transfer	Get one transfers status.
get_list_of_cloud_transfers(transfer_state="ALL")	POST	list_transfer	List transfers by state.
notification_setup(sns_topic=None, sqs_queue=None, emails=None, states=None)	POST	notification_setup	Configure SNS/SQS notifications for cloud transfers.
notification_list()	GET	notification_list	Get notification configuration.
notification_subscribe(emails)	POST	notification_subscribe	Subscribe email addresses to notifications.
notification_unsubscribe(email)	POST	notification_unsubscribe	Unsubscribe an email address.
notification_subscribers()	GET	notification_subscribers	Get list of subscribers.
notification_test(transfer_id=None, state=None, message=None)	POST	notification_test	Send test notification.
notification_enable()	POST	notification_enable	Enable notifications.
notification_disable()	POST	notification_disable	Disable notifications (config preserved).
notification_delete()	POST	notification_delete	Delete notification configuration.

Tasks (/api/tasks/)

Method	HTTP	Endpoint	Returns	Purpose
tasks_get(task_type)	GET	list?task_type=<name>	dict \ None	List tasks. <code>task_type</code> is a <code>TaskType</code> enum.
tasks_reset_stats(task_id, task_type, task_states)	POST	dismiss	Response \ None	Reset/dismiss task stats. <code>task_states</code> may be <code>None</code> (all) or a list of <code>TaskState</code> .

Method	HTTP	Endpoint	Returns	Purpose
<code>tasks_transfer(host_name, src, dst)</code>	POST	<code>transfer</code>	Response \ None	Start a data-transfer task.
<code>start_bryck_report_generate()</code>	POST	<code>capture_bryck_state</code>	Response \ None	Start diagnostic capture.
<code>check_bryck_report_generate()</code>	GET	<code>list?task_type=CAPTURE_BRYCK_STATE</code>	Response \ None	Poll capture status.

Media application (/api/application/)

Method	HTTP	Endpoint	Purpose
<code>add_media(media_type, destination, clip_id, reel_id, media_name, ip_address=None, file_size=None, port=None, session_type=None, payload_type=None, video_format=None, pg_format=None, audio_format=None, audio_sampling=None)</code>	POST	<code>add_media</code>	Add a new media stream.
<code>edit_media(media_id, media_type, destination, clip_id, reel_id, media_name, ip_address=None, file_size=None, port=None, session_type=None, payload_type=None, video_format=None, pg_format=None, audio_format=None, audio_sampling=None)</code>	POST	<code>edit_media</code>	Edit an existing media stream.

Method	HTTP	Endpoint	Purpose
<code>list_media()</code>	GET	<code>list_media</code>	List all media streams.
<code>pause_media(media_id, media_type, ..., pause=True)</code>	POST	<code>pause_media</code>	Pause a stream. Sends <code>media_type</code> as <code>cam_type</code> .
<code>resume_media(media_id, media_type, ...)</code>	POST	<code>resume_media</code>	Resume a paused stream.
<code>remove_media(media_id, media_type, ..., pause=False)</code>	POST	<code>remove_media</code>	Remove a stream.

End-to-end example

```

from session import ApiSession
from bryck_api import BryckApi

with ApiSession.from_login_json("login.json") as sess:
    sess.login()
    api = BryckApi(sess)

    info = api.bryck_info() or {}
    uuid = next(iter(info.get("logical_cards", {})))
    print("Store UUID:", uuid)

    api.scan(uuid)                                # scan the drive tray
    api.format_bryck(
        uuids=[uuid],
        store_type="FILE_STORE",
        raid_level=5,
        IoSize="256",
        DataSync="application sync",
        filestore=True,
        filesystem="zfs",
    )

```

Error handling summary

- Every `BryckApi` method returns `None` if the underlying HTTP call raises. Check for `None` before dereferencing.
- `ApiSession.login()` and `session.get/post` raise on HTTP 4xx/5xx the wrapping `_call()` catches those in `BryckApi`.

- Only `_request_with_retry()` (used when `retry=True`) retries, and only on `ConnectionError`.

3. `ssh_runner.py`

Purpose

Provide a single, reusable SSH/SFTP transport for every runner that needs to run commands on the Bryck OS or push files to it. The module wraps one `paramiko.SSHClient`; every `run()` / `put()` opens a fresh channel over that shared, already-authenticated transport.

Making SSH the sole remote-execution channel is what turns the runners platform-independent: the previous implementation invoked `subprocess` locally (assuming it was executing *on* the Bryck), whereas the new implementation opens an outbound SSH connection **from the machine running the script** so the same code works from the Bryck itself, your workstation, a CI runner, or any container.

Dependencies

- `paramiko >= 3.4.0` (already declared in the projects `pyproject.toml`; install with `pip install paramiko` on hosts that lack it).

Module-level constants

Name	Value	Meaning
<code>DEFAULT_KEY_FILE_REMOTE_PATH</code>	<code>"/opt/bryck/bryckapi/downloads/Keyfile"</code>	Bryck OS server destination for uploaded encryption key files.
<code>DEFAULT_SSH_PORT</code>	<code>22</code>	SSH port used by <code>SshRunner</code> .

Class: `SshRunnerError`

Raised for every transport-level failure (`paramiko.SSHException`, `OSError`, missing credentials). Wraps the underlying exception via `__cause__`.

Class: `SshRunner`

```
from ssh_runner import SshRunner

with SshRunner(host="192.168.6.32",
```

```

        username="bryck",
        password="<password>") as ssh:
    rc, out, err = ssh.run("uname -a")
    ssh.put("./keyfile", "/opt/bryck/bryckapi/downloads/keyfile")

```

Constructor

```
SshRunner(host, username, password, port=22, timeout=15) -> None
```

- Does **not** connect at construction time call `connect()` (or use the context manager).
- Raises `SshRunnerError` immediately if `host`, `username`, or `password` is empty / falsy.

Factory

```
SshRunner.from_session(session: ApiSession, port=22, timeout=15) ->
    SshRunner
```

Convenience constructor that reuses the `host` from `session.host` and the SSH credentials stashed on the session by `ApiSession.from_login_json(session.ssh_username, session.ssh_password)`. Raises `SshRunnerError` if the session was constructed without `bryckserver_*` credentials.

Public methods

Method	Purpose
<code>connect()</code>	Open the SSH transport (idempotent). Uses <code>paramiko.AutoAddPolicy()</code> ; <code>allow_agent=False</code> and <code>look_for_keys=False</code> .
<code>close()</code>	Close the transport (safe to call twice).
<code>run(cmd, timeout=60)</code>	Execute <code>cmd</code> on the remote host via a new channel. Returns <code>(returncode, stdout, stderr)</code> . Returncode <code>-1</code> means the channel timed out. <code>cmd</code> is a single string interpreted by the remote users shell.
<code>put(local_path, remote_path)</code>	SFTP upload. Raises <code>SshRunnerError</code> on failure.

Context manager Both `__enter__` (`connect()`) and `__exit__` (`close()`) are implemented, so with `SshRunner(...)` as `ssh`: is the recommended form in short-lived scripts.

Security notes

- **Password auth only.** SSH-agent forwarding and on-disk private keys are explicitly disabled on the client side to make behaviour deterministic across machines.
- **Auto-accepted host keys.** `AutoAddPolicy` trusts unknown host keys on first use and appends them to the runner users `~/.ssh/known_hosts`. For strict deployments, swap the policy for `paramiko.RejectPolicy` and pre-populate `known_hosts`.
- **Command strings, not argv lists.** `run()` uses `exec_command`, which spawns the remote users default shell. Pass a plain command line and `shlex.quote()` any untrusted input.

Typical use inside a runner

```
session = ApiSession.from_login_json("login.json")
ssh = SshRunner.from_session(session)
try:
    session.login()
    ssh.connect()
    # ... REST calls via BryckApi(session) ...
    # ... SSH validators via ssh.run(...) ...
finally:
    ssh.close()
    session.close()
```